



NanoFlow: Towards Optimal Large Language Model Serving Throughput

Kan Zhu, Yufei Gao , Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu,
Dedong Xie, Qinyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori,
Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, Baris Kasikci

LLMs Enable Many New Applications

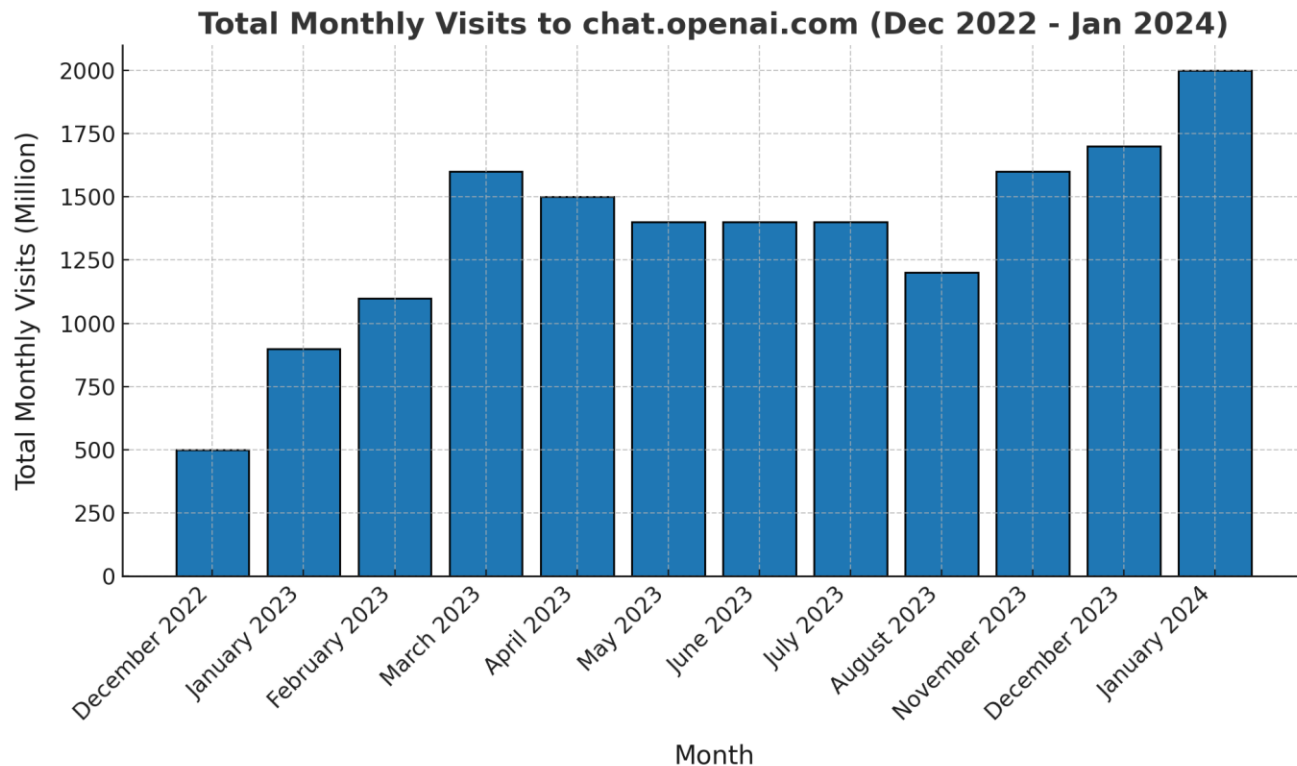
AI Assistants (ChatGPT, Google Bard)  

Text-to-Image (DALL·E, MidJourney)  

Creative Writing (Jasper, Copy.ai)  

AI Coding Tools (GitHub Copilot, Replit)  

LLM Serving Systems Face Surging Demand



Cost of Serving LLMs is Massive

NVIDIA H100 HGX Server

- ~ \$200,000
- ~ 8,000W
- High operating cost

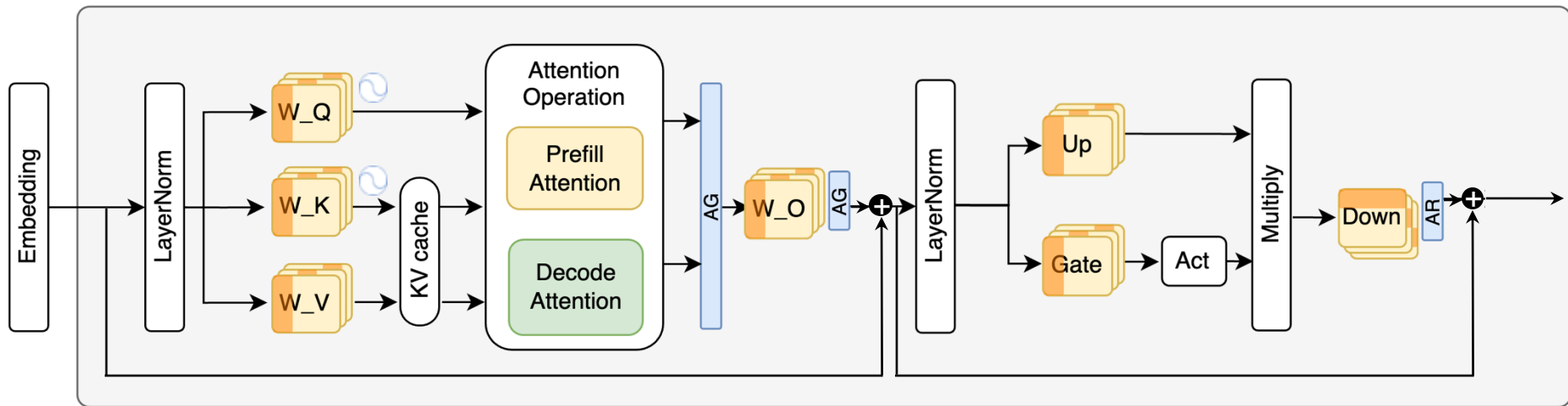


[1] [Tom's Hardware](#)

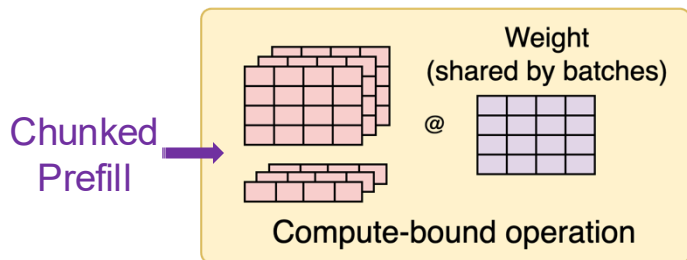
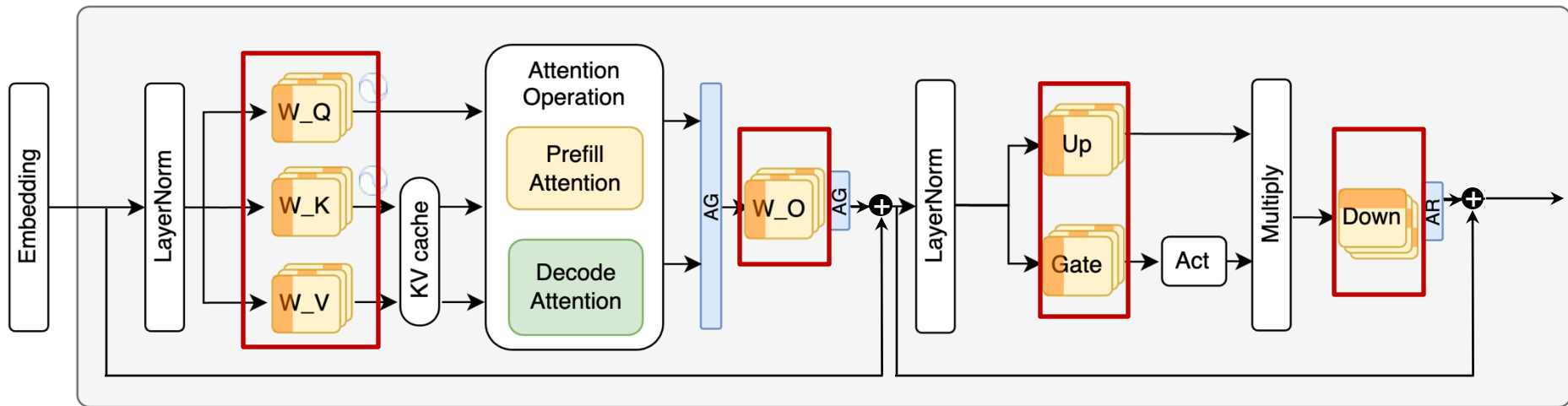
[2] [The Register](#)

Achieving high LLM serving throughput is key for cost reduction

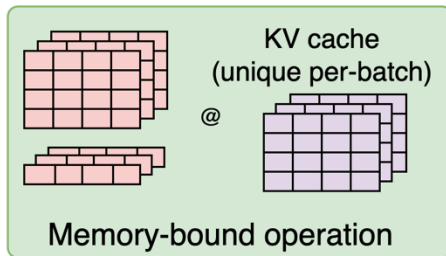
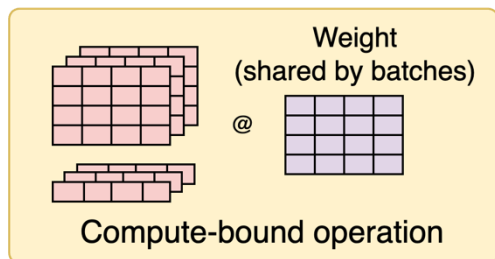
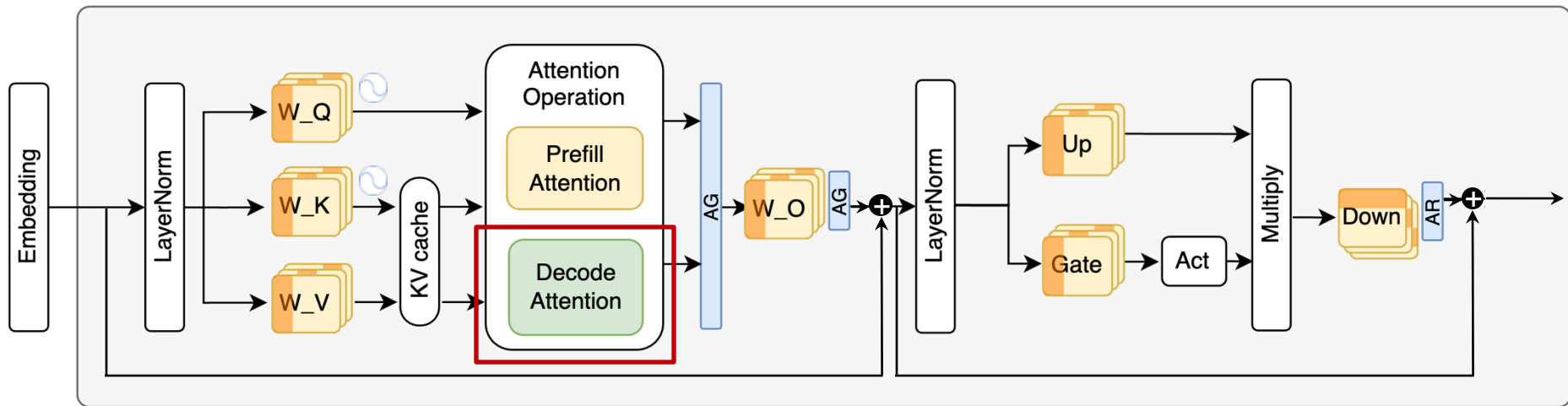
Decoder-Only Transformer



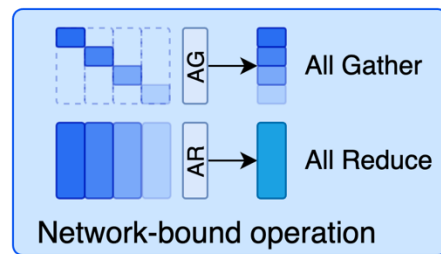
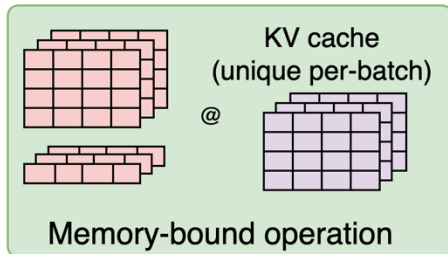
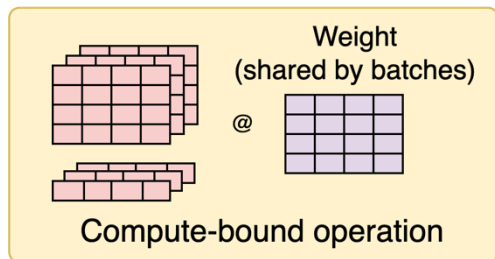
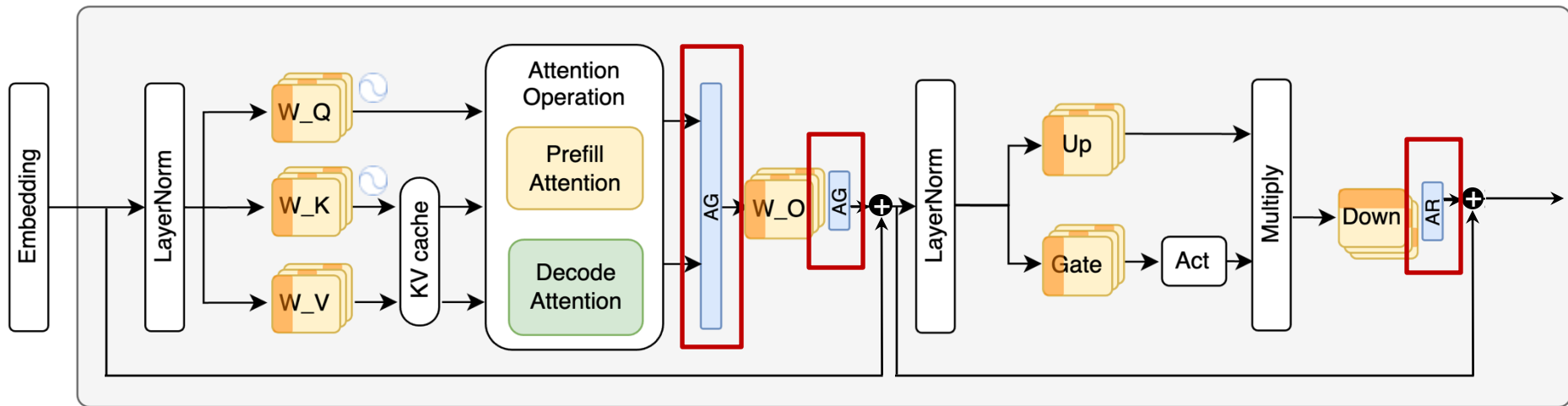
Decoder-Only Transformer



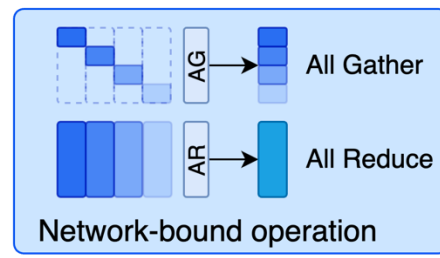
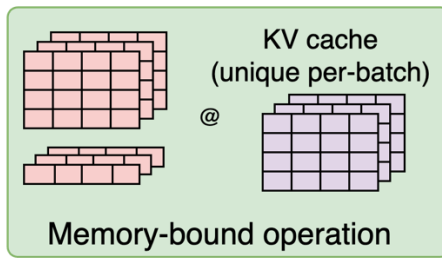
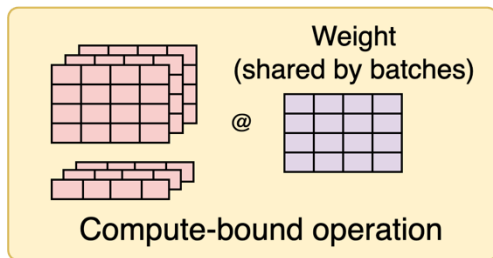
Decoder-Only Transformer



Decoder-Only Transformer



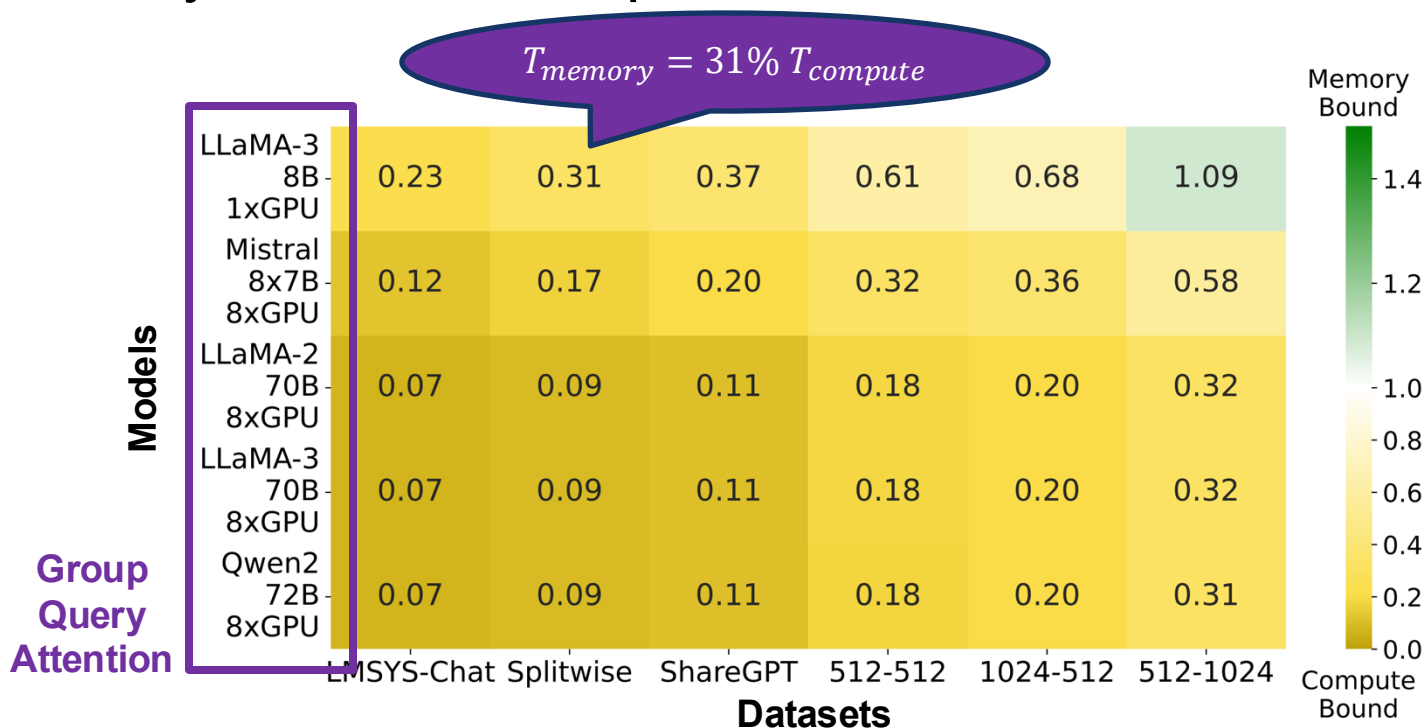
Heterogenous Resource Requirements in LLM Serving



Which Resource is the Key Bottleneck for LLM Serving?

Calculate and compare the time for GPU to deliver each resource for the workload

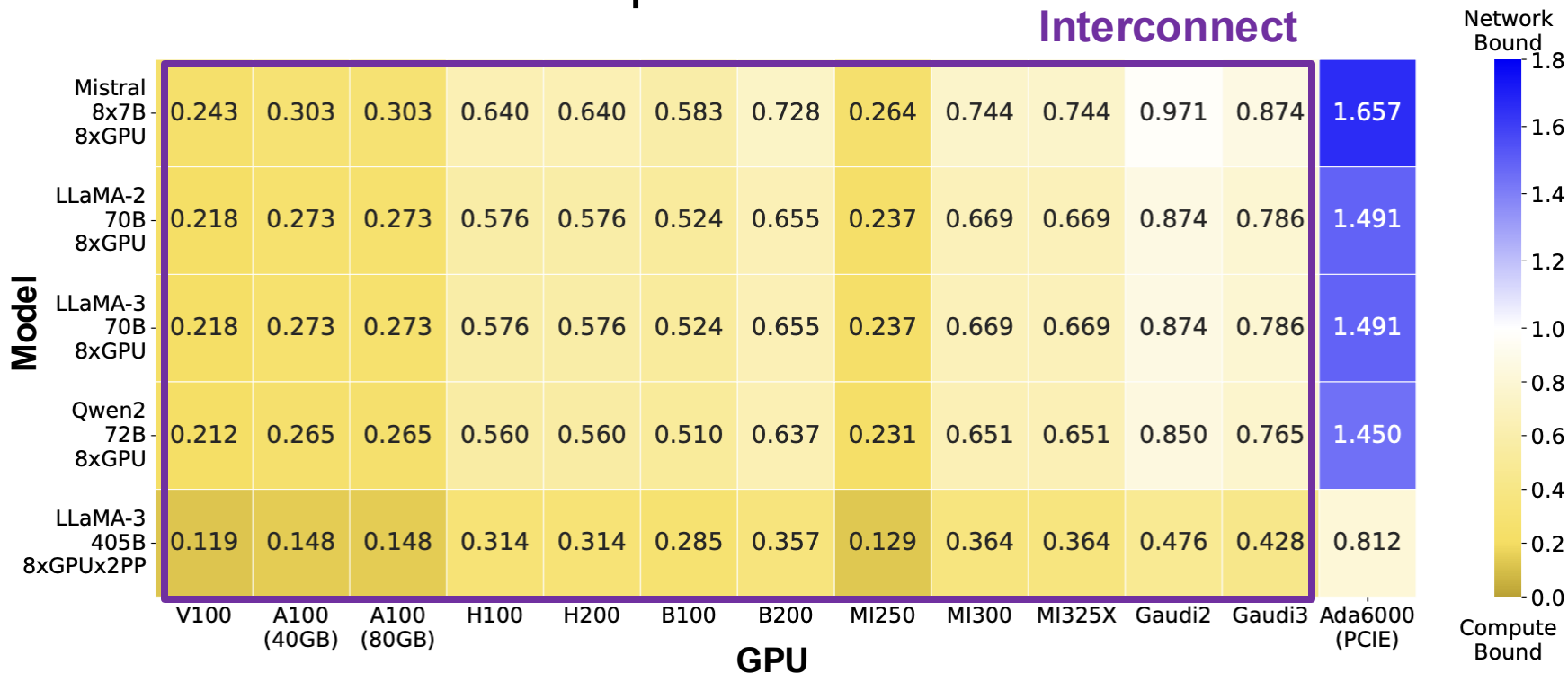
Memory Time vs Compute Time



Compute is more bottlenecked than Memory

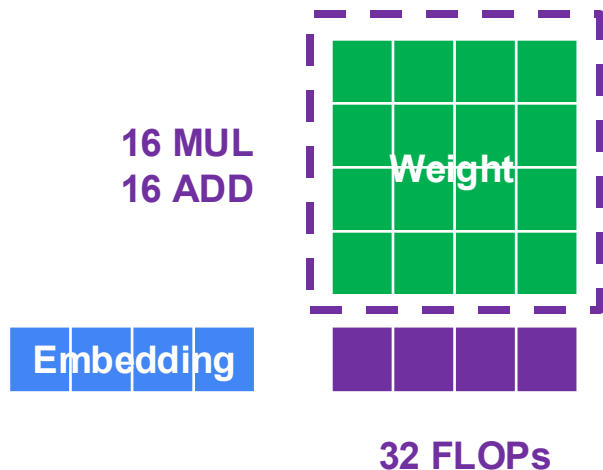
Network Time vs Compute Time

High Bandwidth
Interconnect

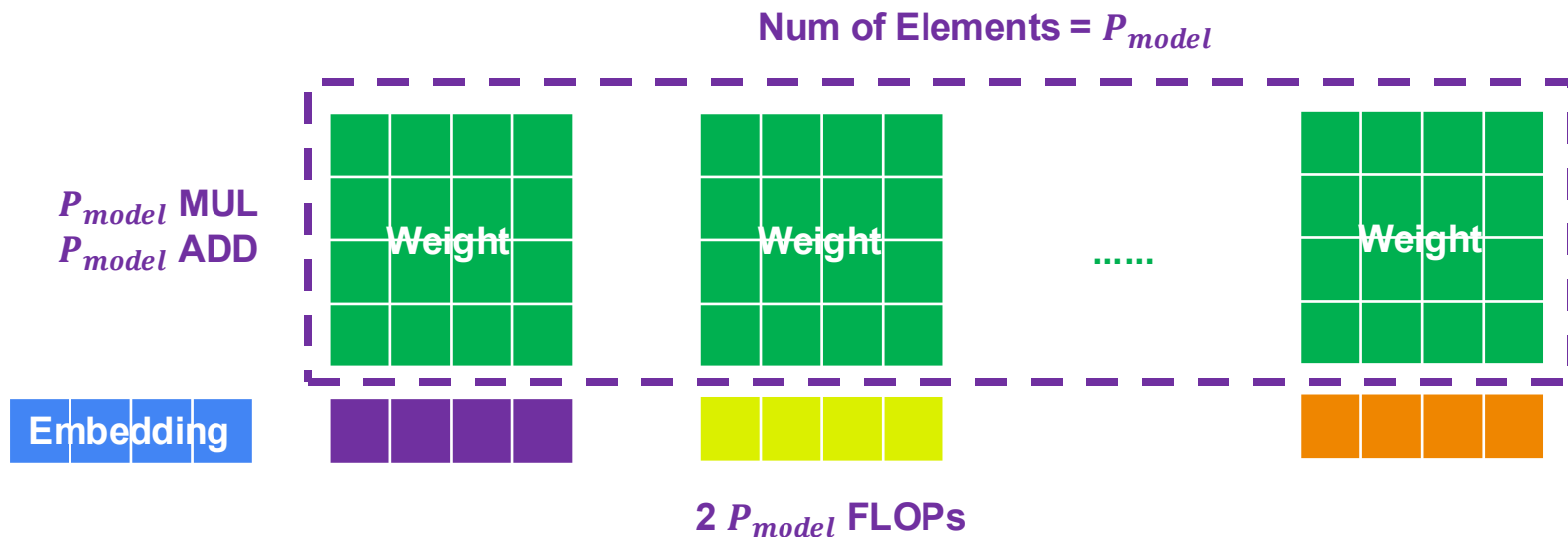


Compute the most constraint resources

Optimal Throughput When Being Compute Bound



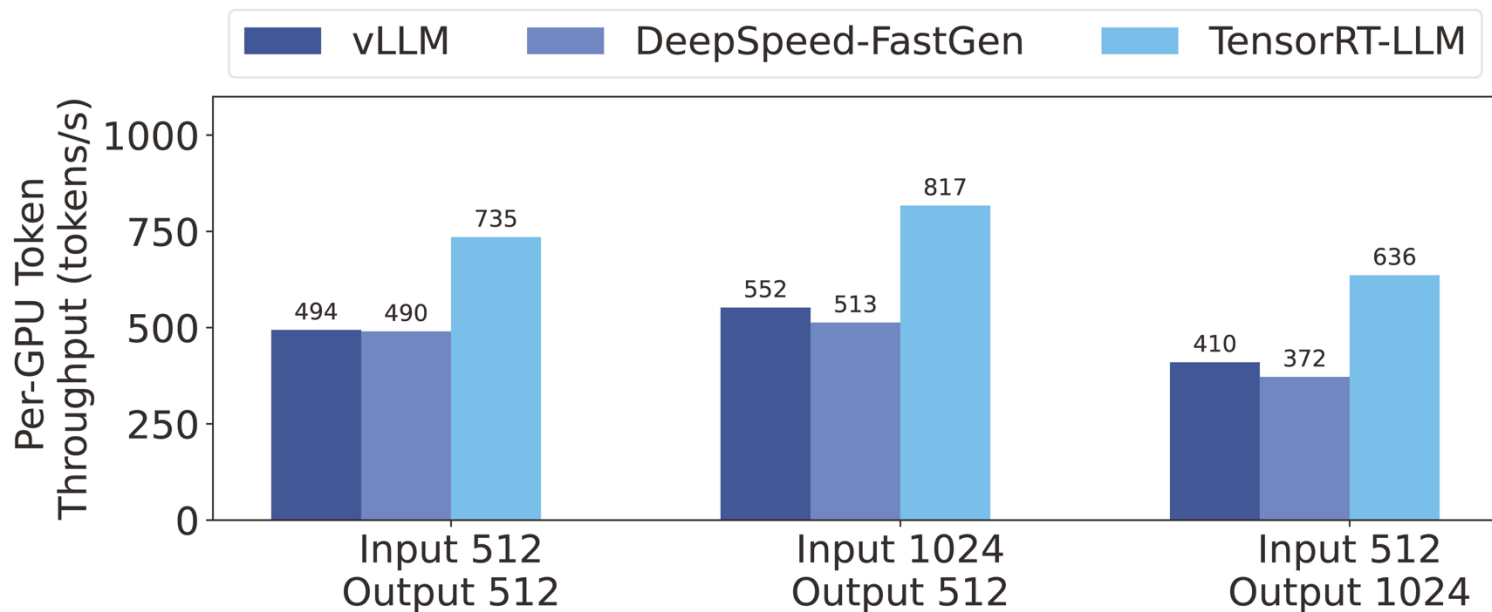
Optimal Throughput When Being Compute Bound



$$Throughput = \frac{GPU\ Compute}{2P_{model}}$$

e.g., LLaMa3 70B, A100 1857 token/s/GPU

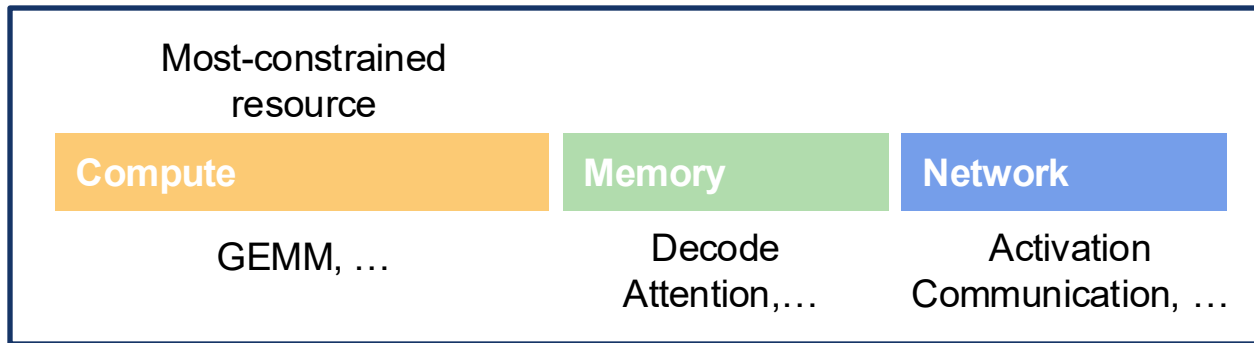
Performance of Existing Frameworks



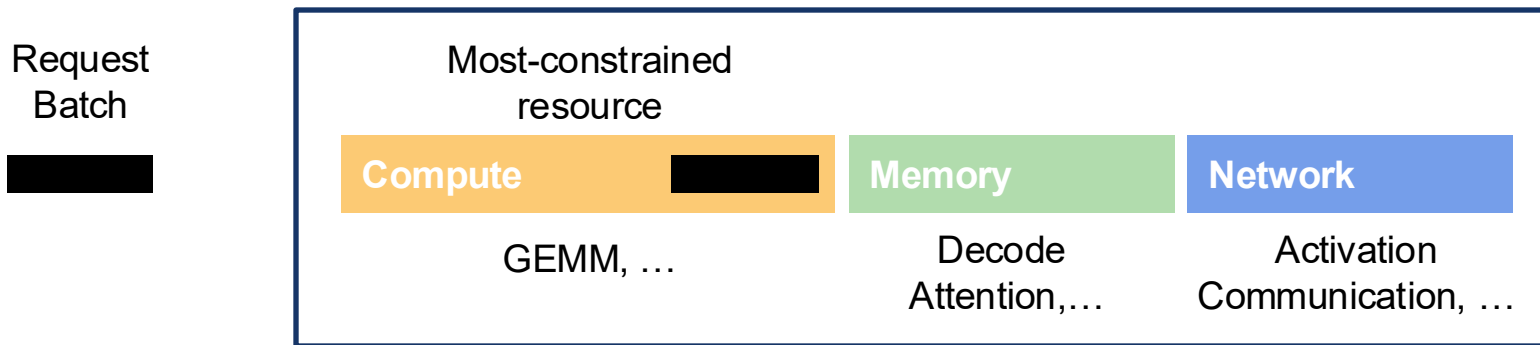
There is a large gap to optimal throughput

Existing LLM Inference Engines

Request
Batch

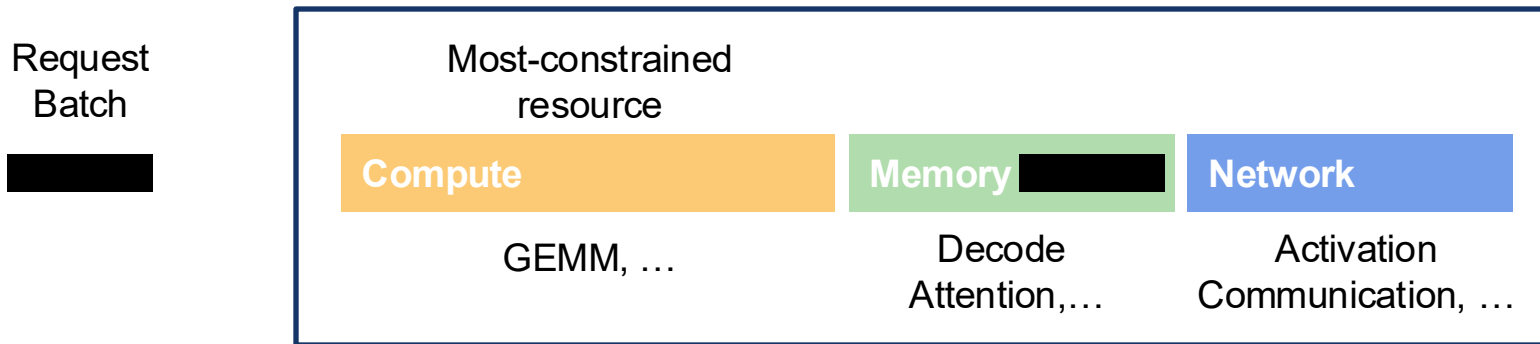


Existing LLM Inference Engines



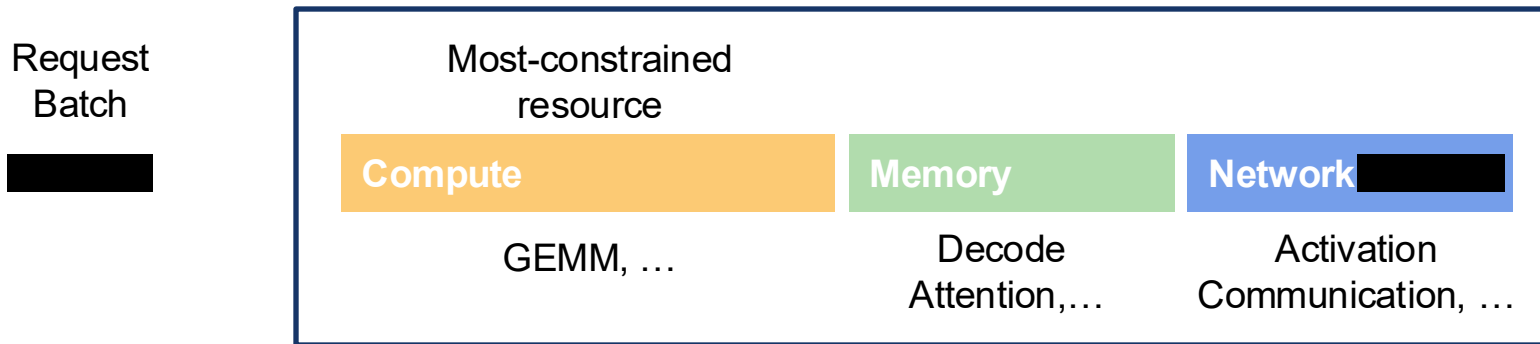
Batches processed sequentially by heterogeneous operations

Existing LLM Inference Engines



Batches processed sequentially by heterogeneous operations

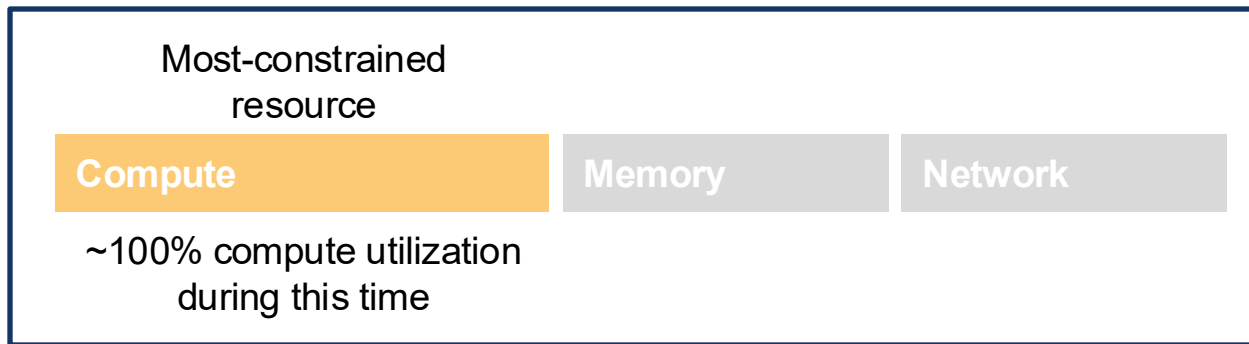
Existing LLM Inference Engines



Batches processed sequentially by heterogeneous operations

Existing LLM Inference Engines

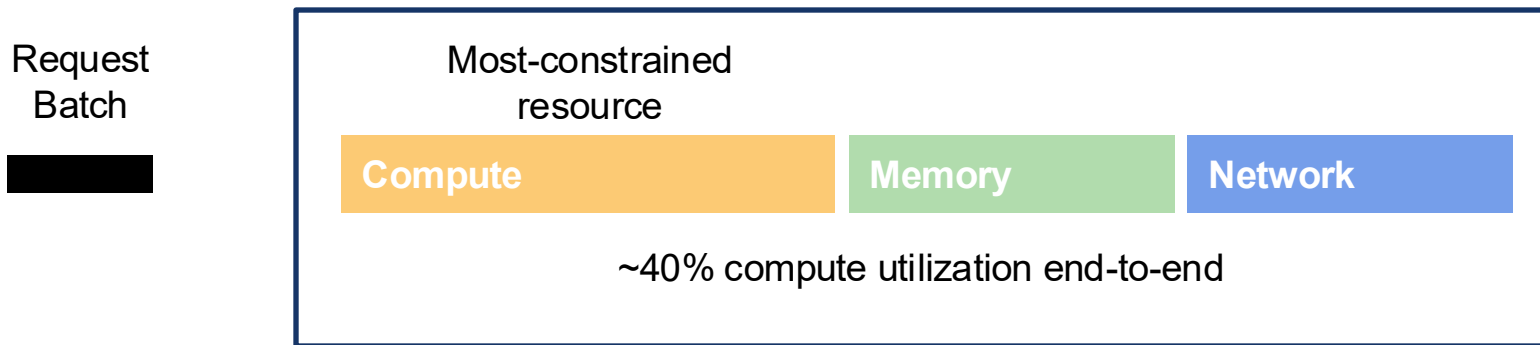
Request
Batch



Batches processed sequentially by heterogeneous operations

- Operations have high resource utilization locally
 - Full access to GPU
 - ~100% compute utilization during compute-bound operations

Existing LLM Inference Engines

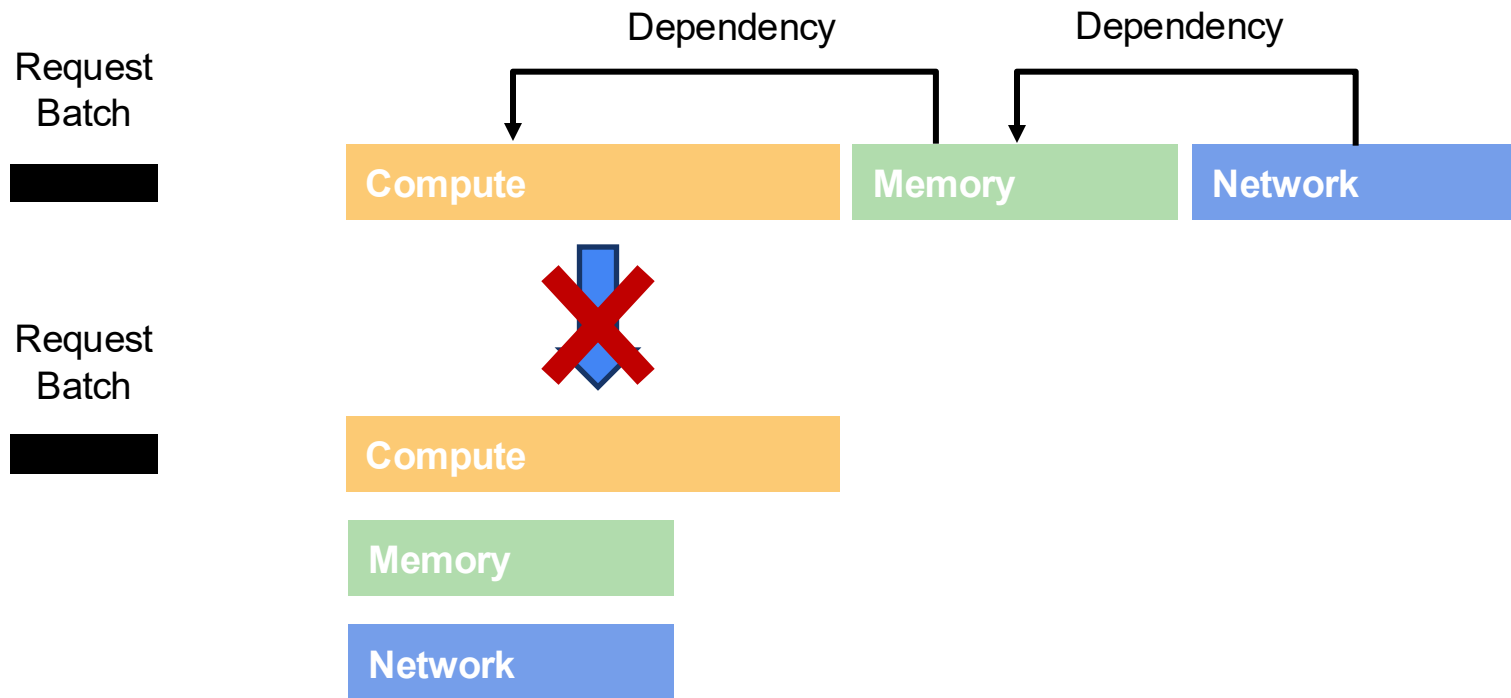


Batches processed sequentially by heterogeneous operations

- Operations have high resource utilization *locally*
 - Full access to GPU
 - ~100% compute utilization during compute-bound operations
- End-to-end resource utilization is low
 - 40% compute utilization end-to-end

Increase the compute utilization is the key for filling the throughput gap

Direct Overlapping is Infeasible



Request
Batch



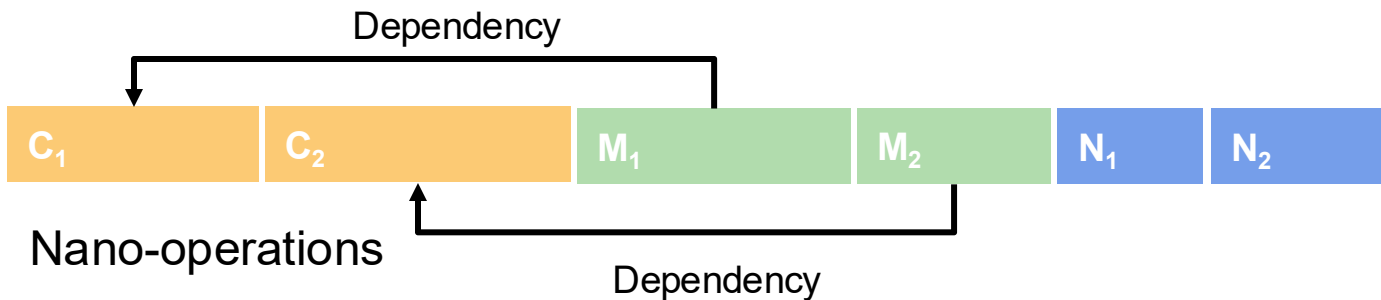
1024 tokens



Nano-batches



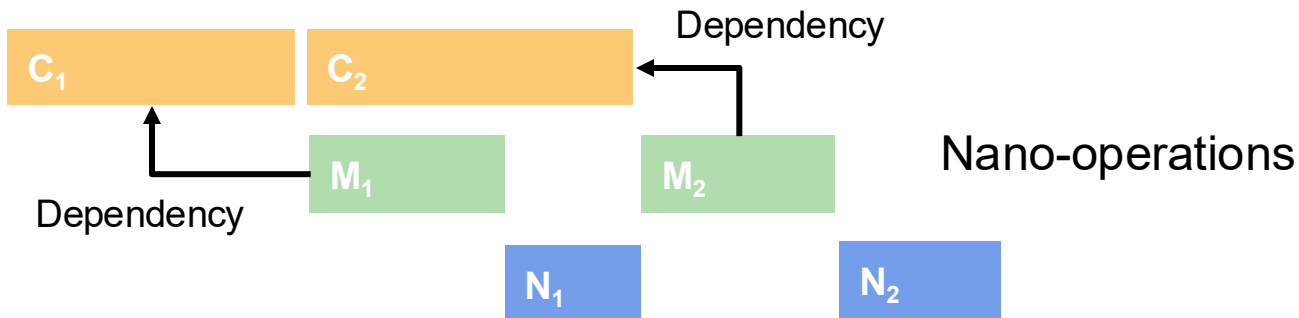
512+512
tokens



Request
Batch



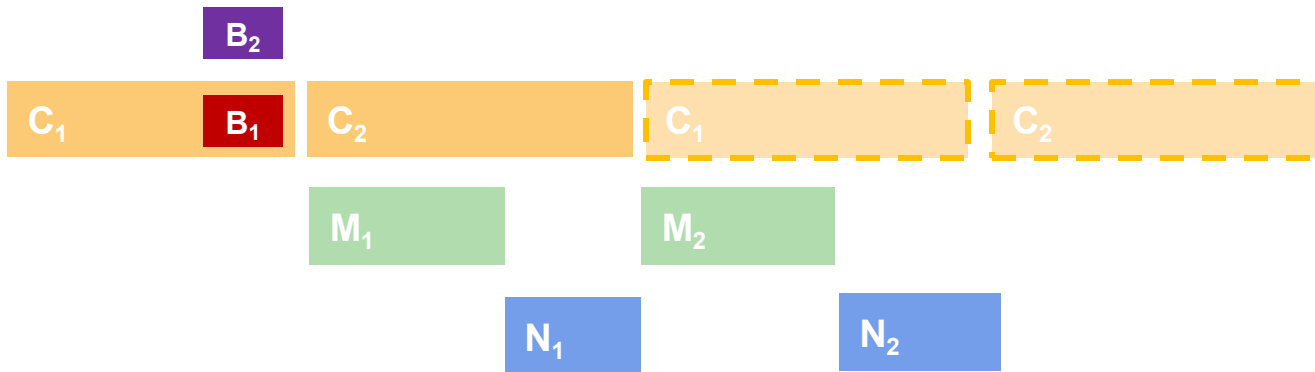
Nano-batches



Request
Batch



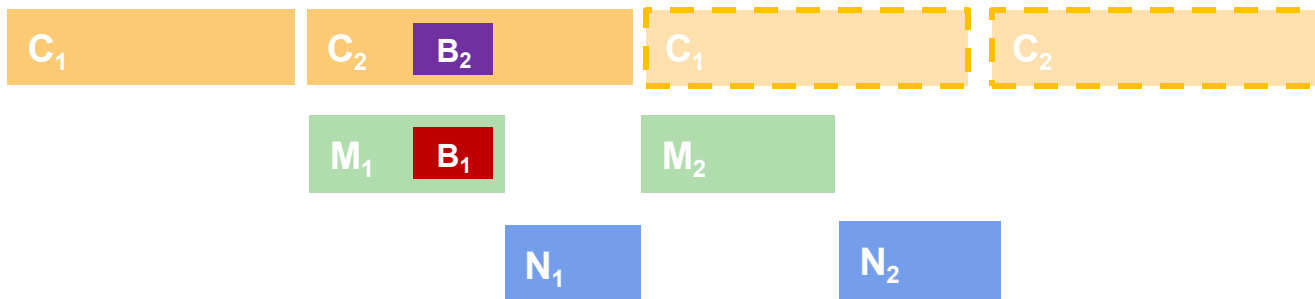
Nano-batches



Request
Batch



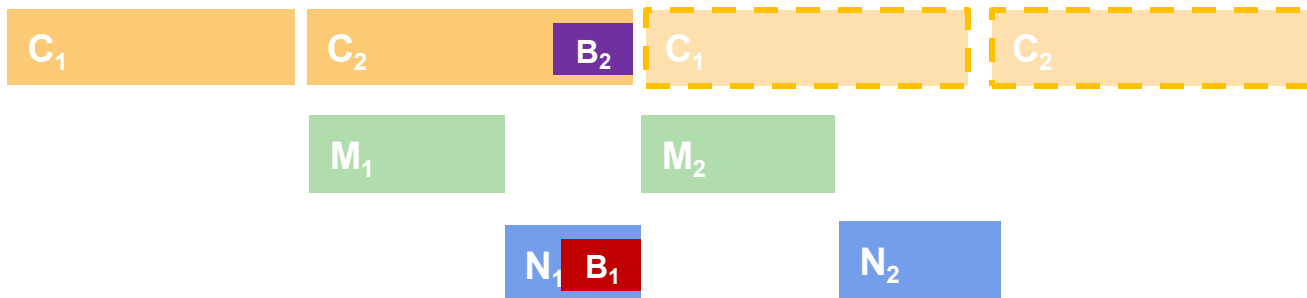
Nano-batches



Request
Batch



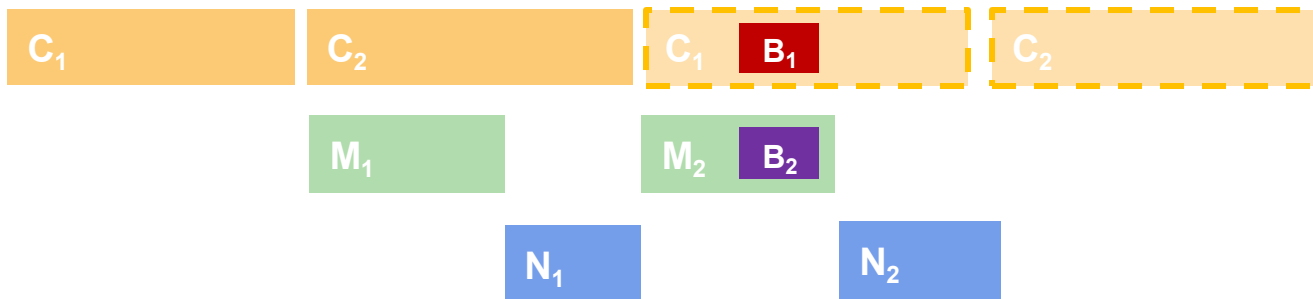
Nano-batches



Request
Batch



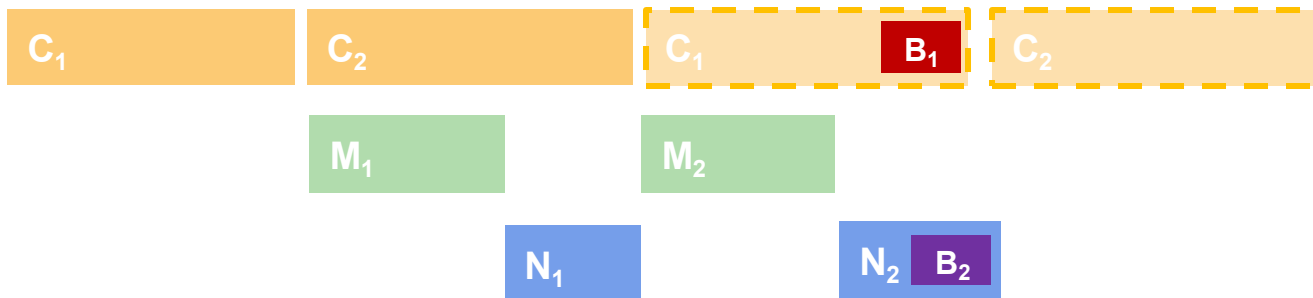
Nano-batches



Request
Batch



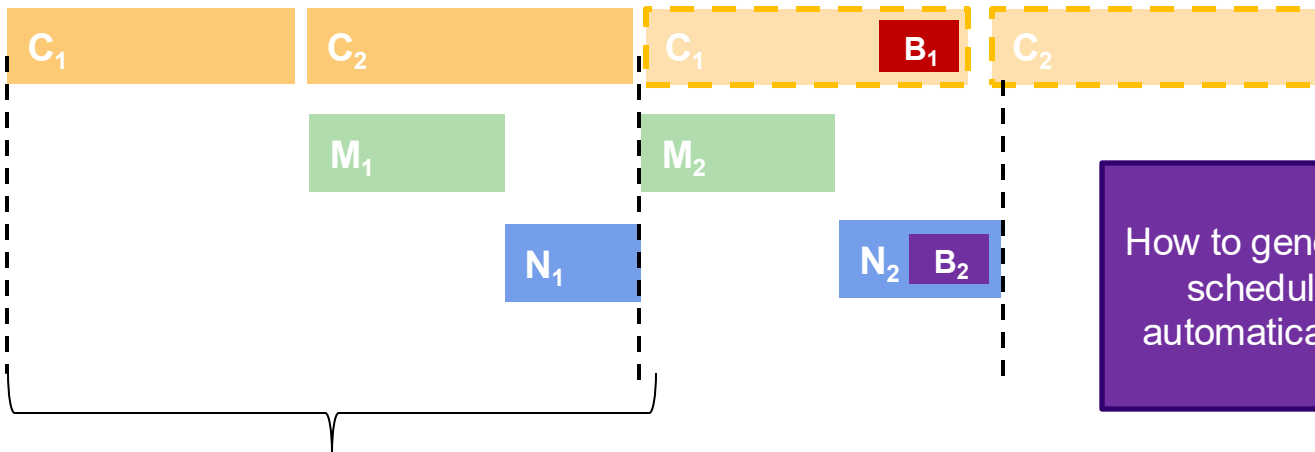
Nano-batches



Request
Batch



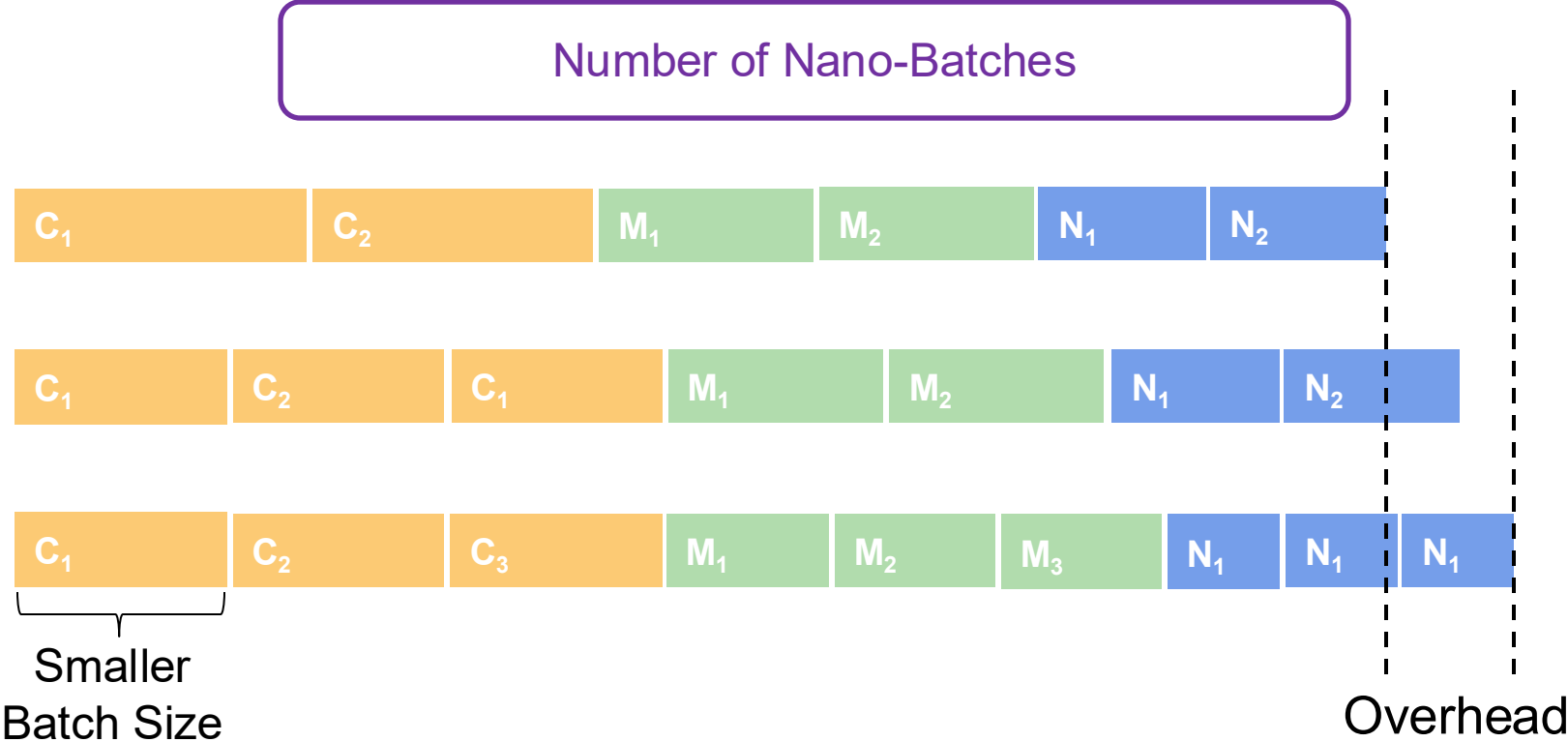
Nano-batches



Increased end-to-end compute utilization

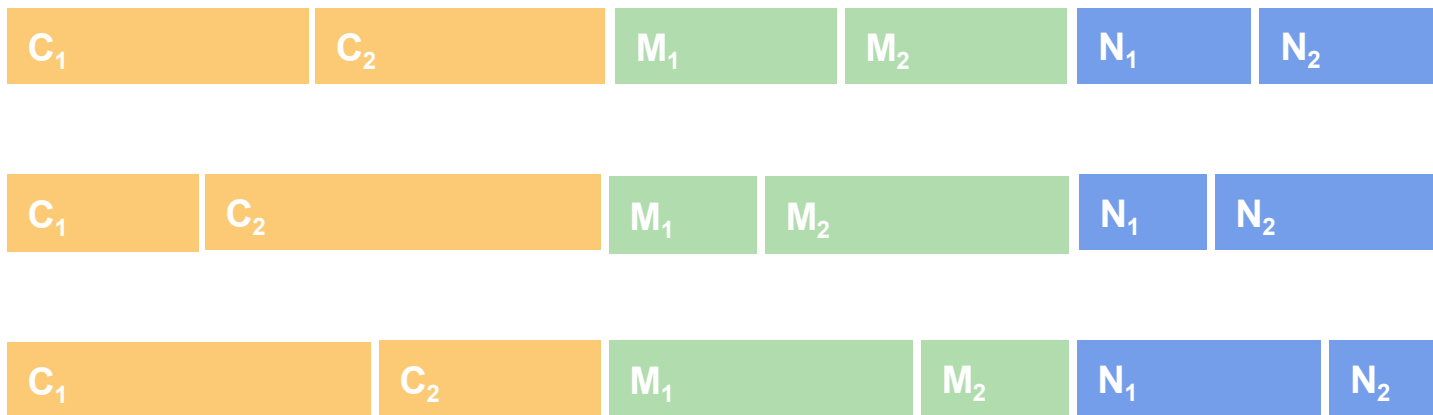
How to generate
schedule
automatically?

Nano-batch Design Space



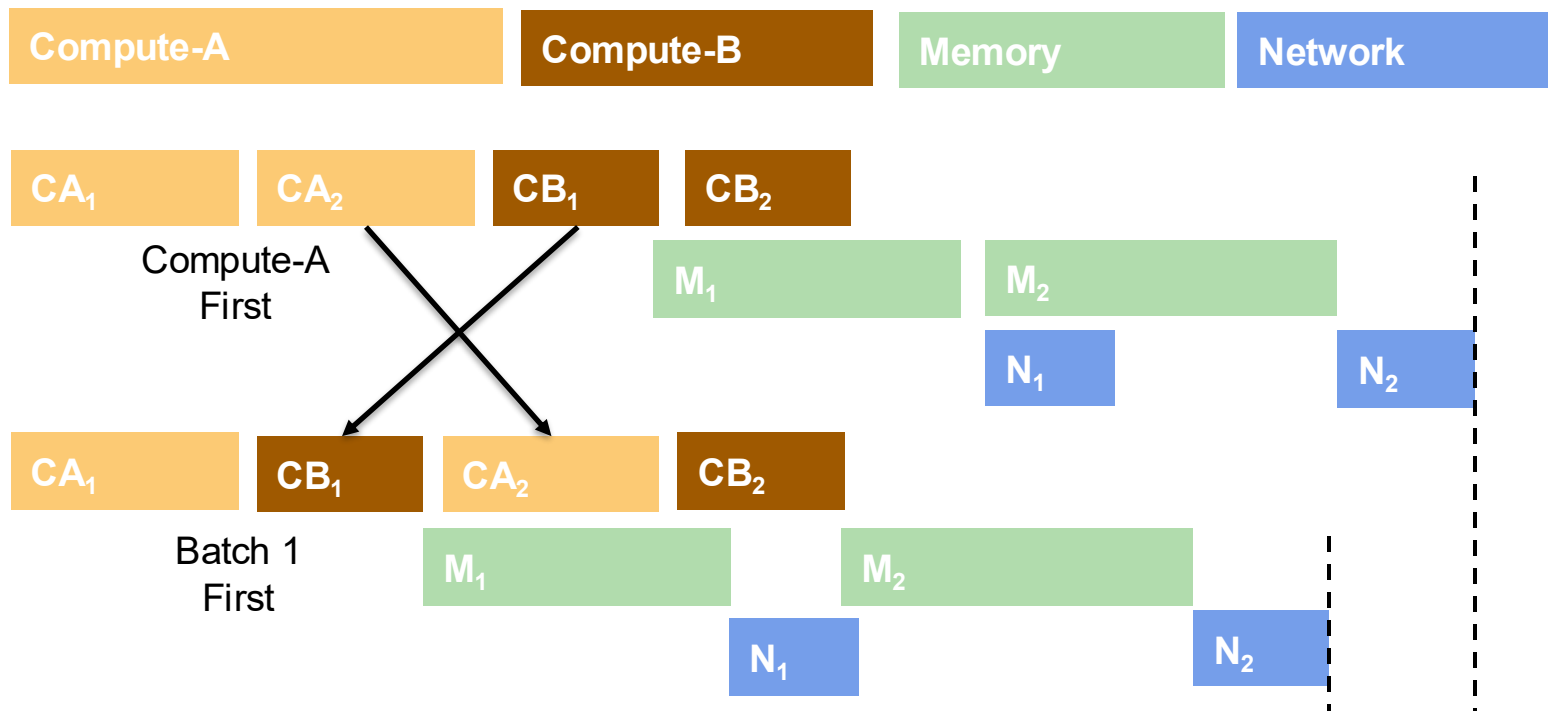
Nano-batch Design Space

Size of Nano-Batches

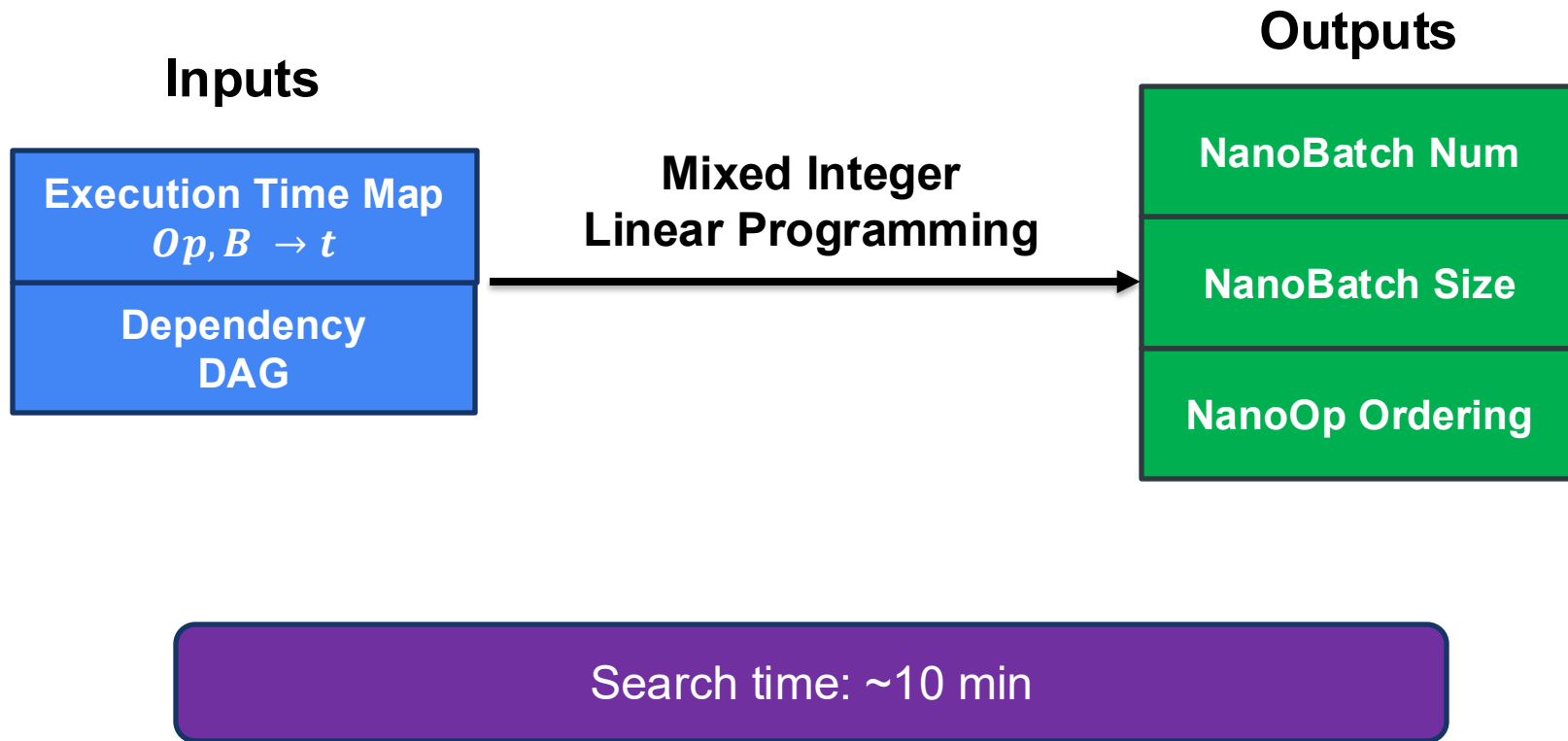


Nano-batch Design Space

Ordering of Nano-Operations



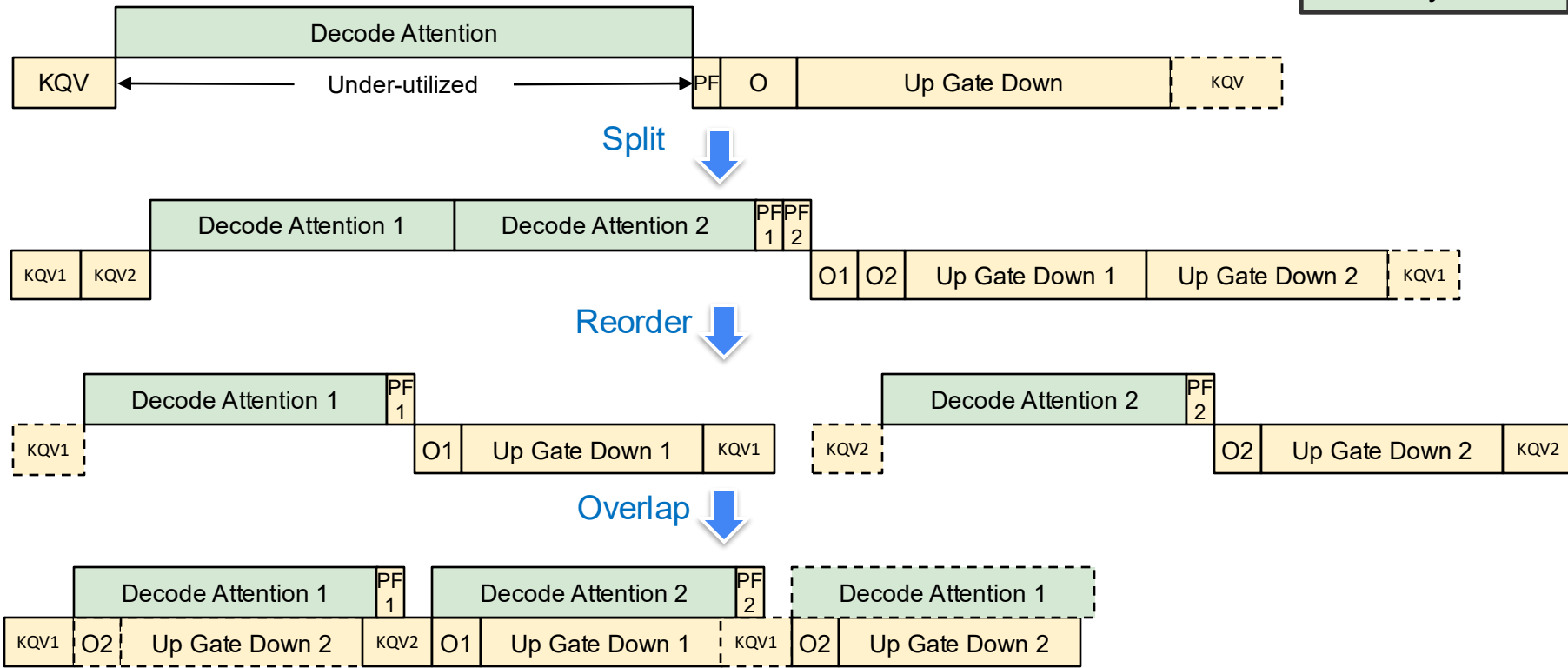
Automated Pipeline Search – Phase 1



8B Pipeline Design

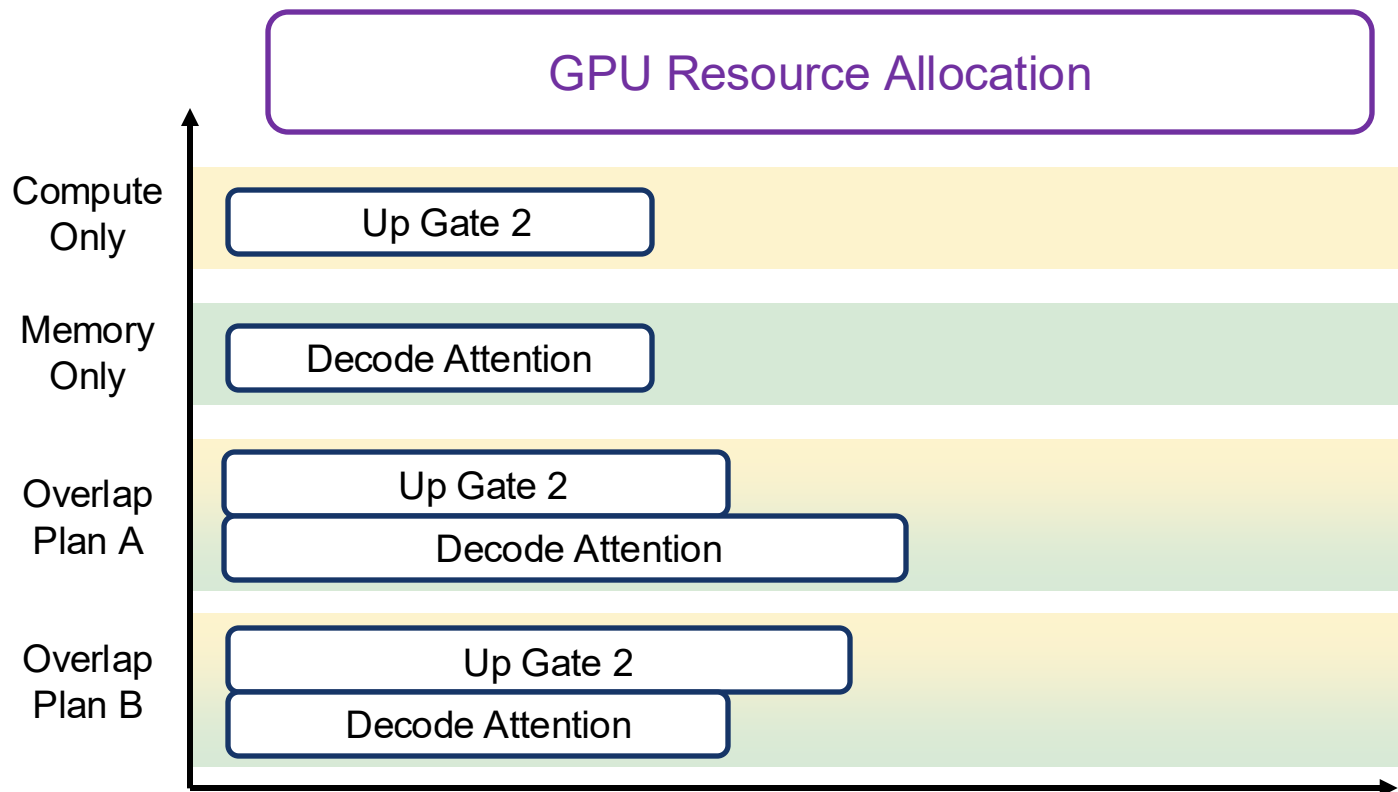
Compute-bound

Memory-bound



Compute-bound kernels fully occupy the GPU

Overlapping Operations Degrades Performance



Automated Pipeline Search – Phase 2

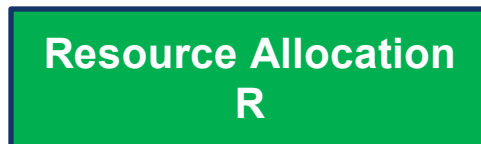
Inputs



Mixed Integer
Linear Programming



Outputs



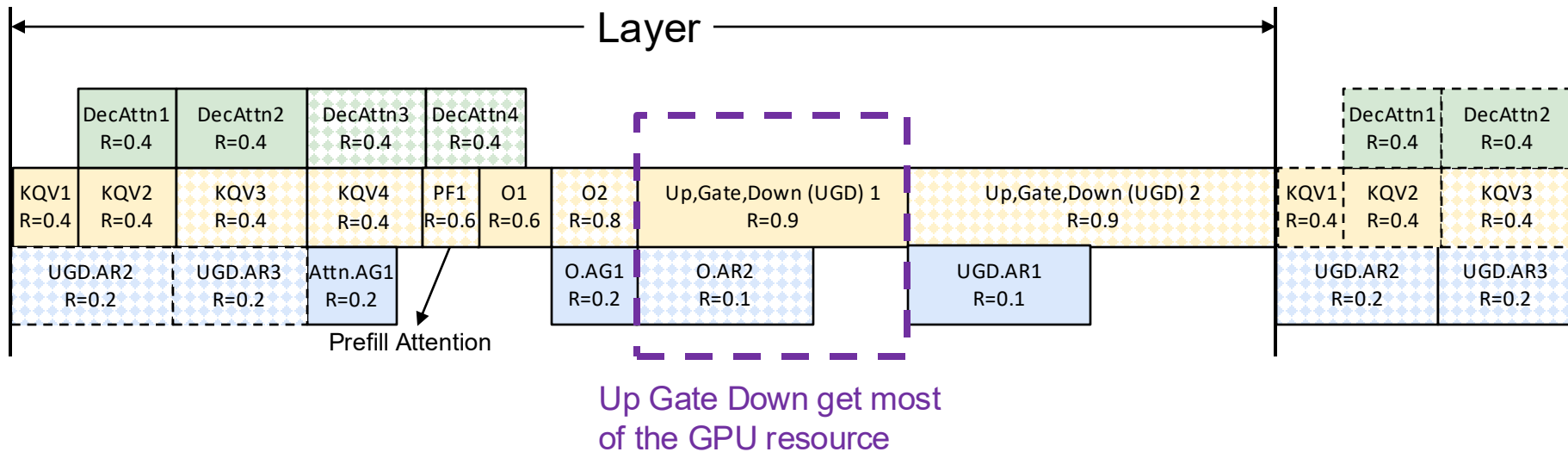
Search time: ~30 min

70B – Nanoflow Pipeline

Compute-bound

Memory-bound

Network-bound



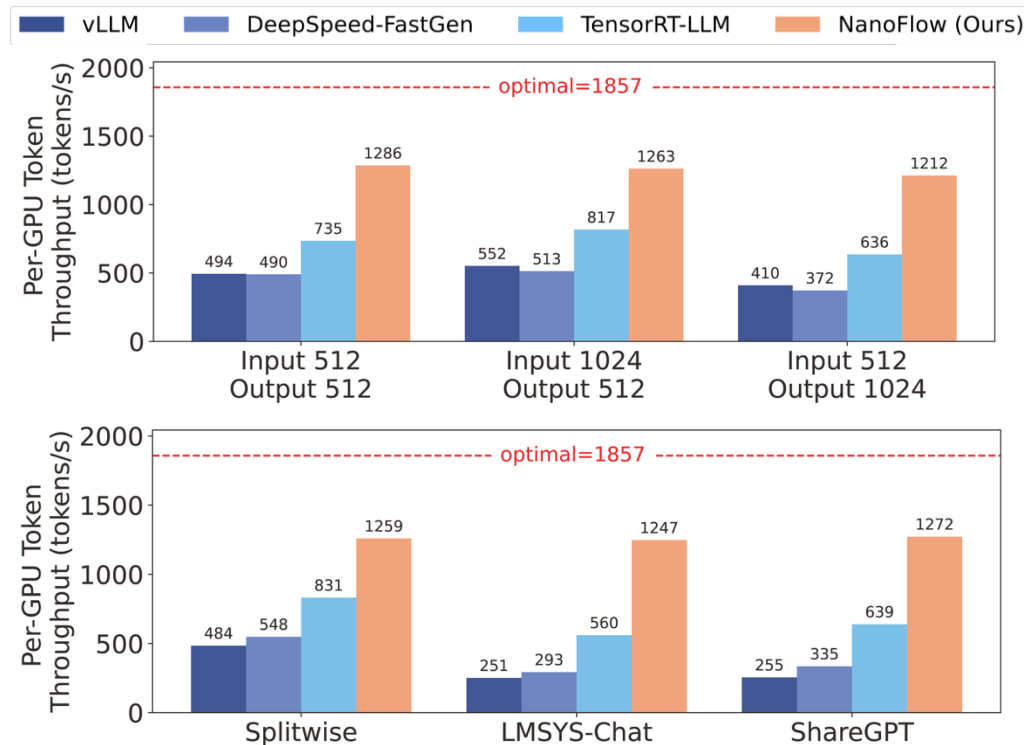
Evaluation: End-to-End Throughput

Dataset	Avg. Input (Std)	Avg. Output (Std)
Splitwise [34]	1155 (1109)	211 (163)
LMSYS-Chat-1M [52]	102 (169)	222 (210)
ShareGPT [1]	246 (547)	322 (244)

1.73x const length I/O throughput gain

1.91x dataset throughput gain

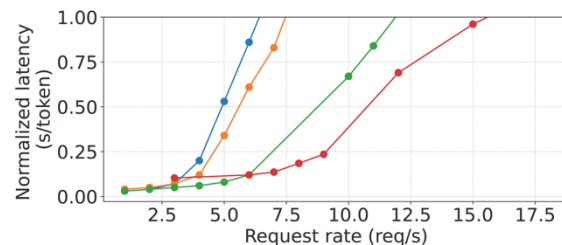
68.5% of optimal throughput



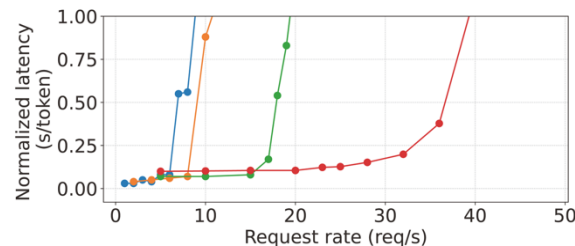
Evaluation: Latency

Similar (but a bit higher) latency at low request rate

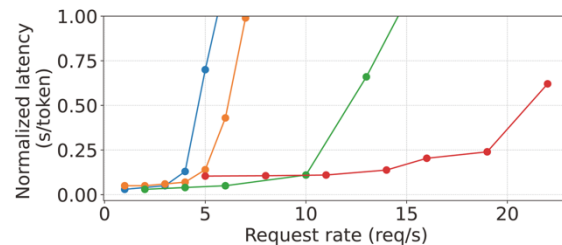
Processes 1.64x requests at 200ms SLO



Splitwise

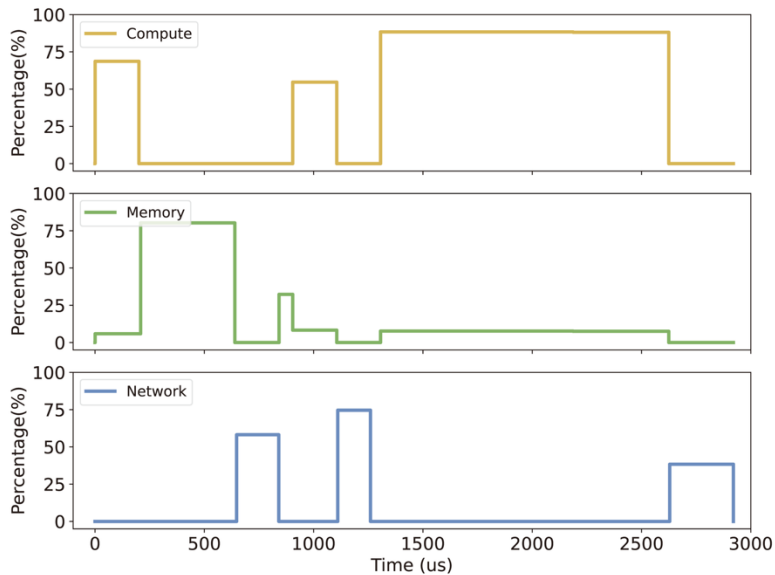


Mlsys

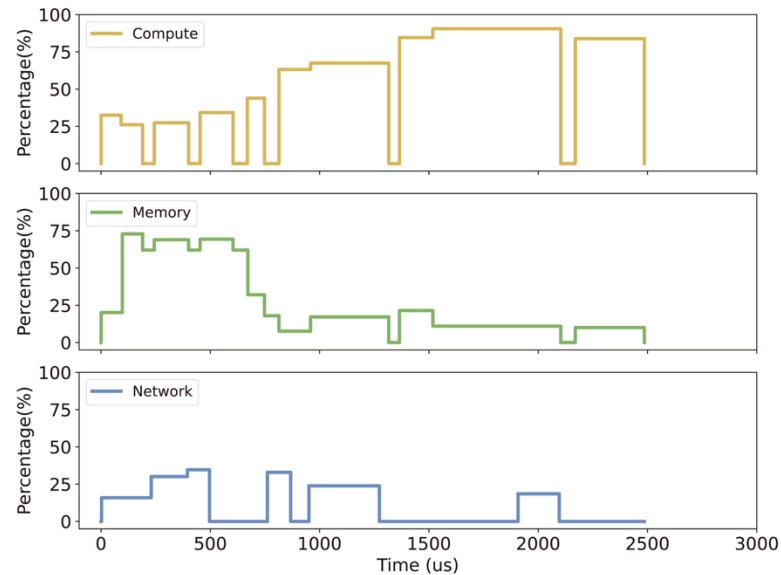


ShareGPT

Evaluation - Resource usage



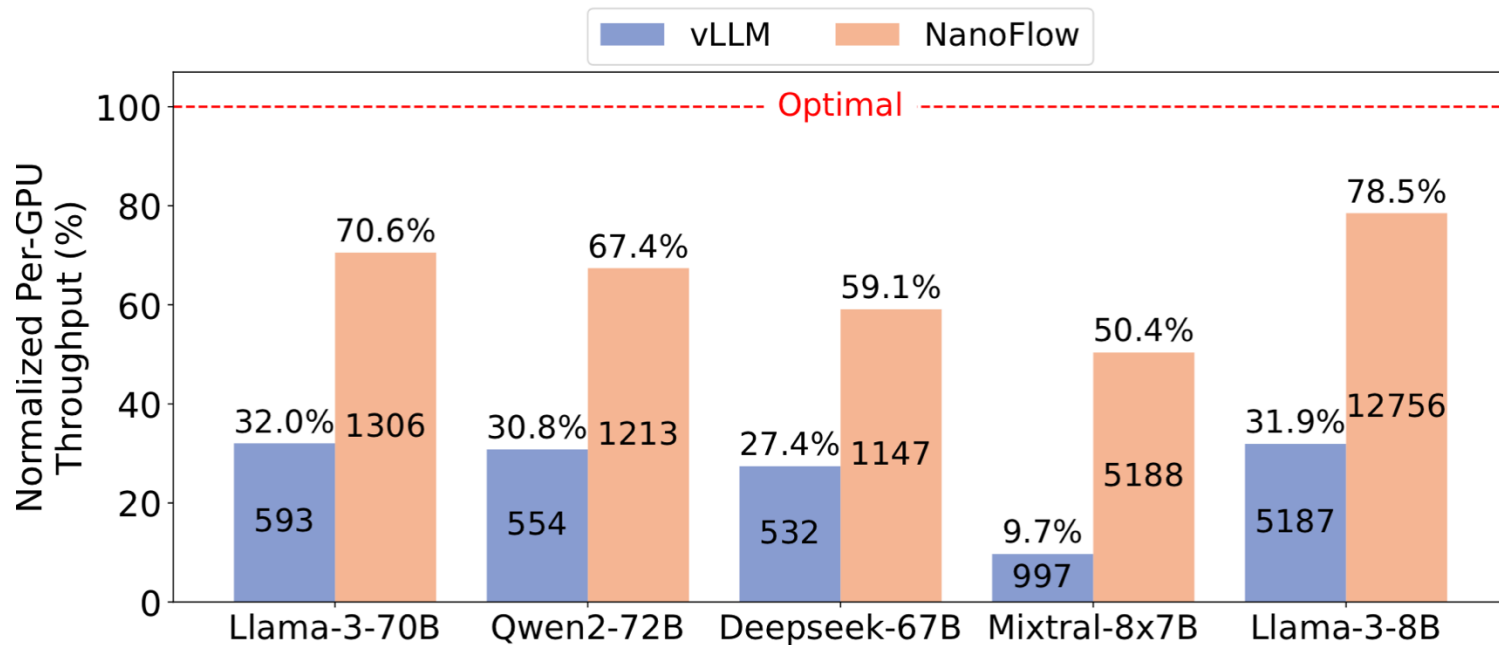
Non-overlap baseline



Nanoflow

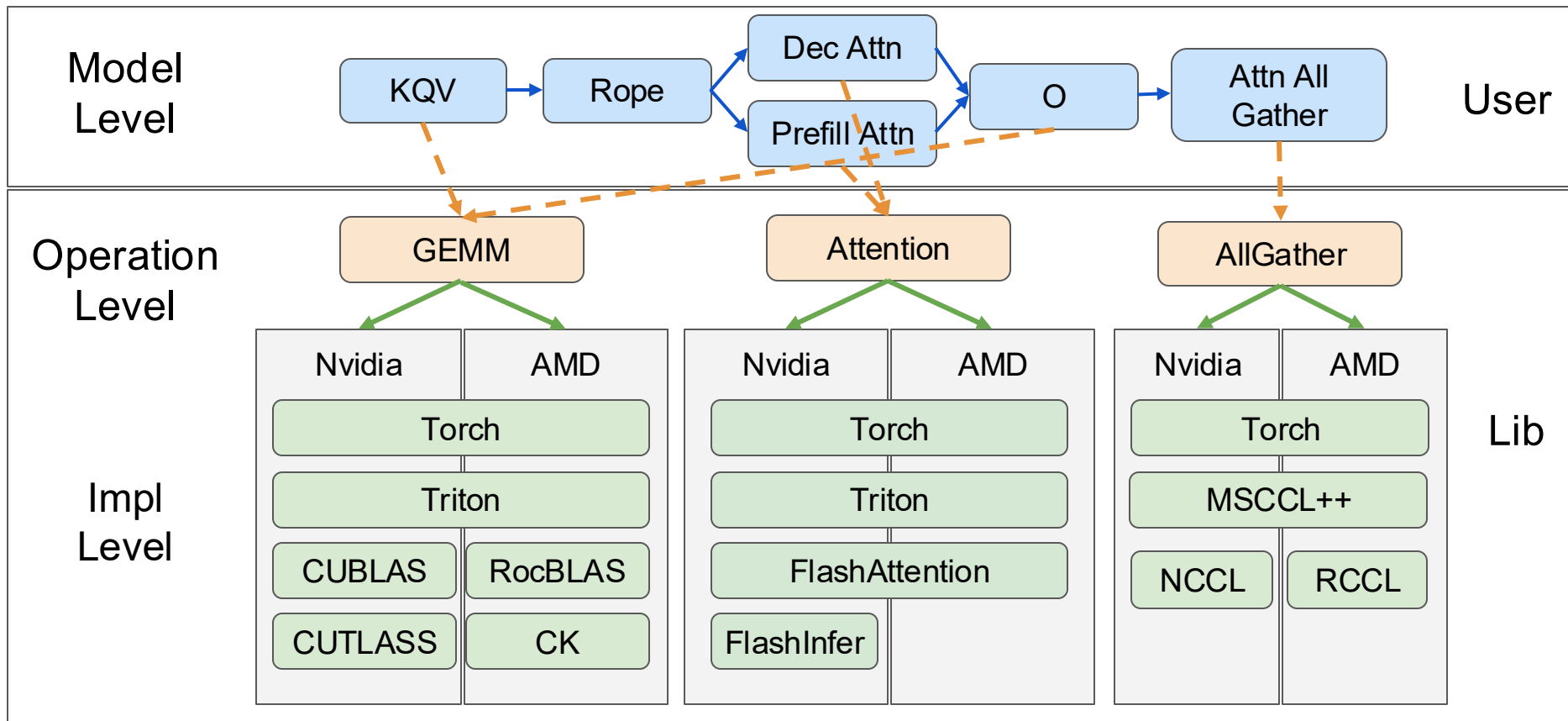
Nanoflow utilizes multiple resources concurrently, achieving higher average compute usage 41

Evaluation – Nanoflow Pipelines with Different Models



Porting Nanoflow to other models demonstrates 50.4% to 78.5% of optimal throughput

NanoFlow as a Research Oriented Serving Engine



NanoFlow as a Research-Oriented Serving Engine

Small Code Base

- Only Representative Models
- Lower Backward Compatibility Pressure

Performance Centric

- Support Custom Kernels
- Automatically Search for the Best Implementation

Heterogenous Support

- CPU-GPU, GPU-GPU Cooperation
- Assist Pipeline Design with Auto-search



Paper



Nano-batching to Overlap Heterogeneous Operations

Automated Search and Efficient Runtime

GitHub



1.91x throughput gain

50.4-78.5% optimal throughput



kanzhu@cs.washington.edu

